

Dossier de Projet

**Titre Professionnel : Concepteur Développeur d'Applications
(CDA)**

Projet : Promptimo

SaaS de gestion et de prospection pour agents immobiliers

Candidat : LE PLAT Jonathan

Date de naissance : 27/07/1991

Organisme de formation : La Plateforme_, Marseille

Année : 2026

Tableau de compétences — RNCP 37873

Bloc 1 — Développer une application sécurisée

CP1 | Installer et configurer son environnement de travail

Configuration Next.js 16 TypeScript strict, Prisma v7, PostgreSQL 18 sur VPS Ionos Ubuntu 26.04 (PM2, Nginx, nvm, pnpm)

CP2 | Développer des interfaces utilisateur

Pages React (RSC + Client Components) : liste clients/biens, formulaires CRUD, composant SSE avec curseur clignotant, Tailwind v4 + shadcn/ui

CP3 | Développer des composants métier

Magic link (flux complet SHA-256 + JWE), webhook Stripe atomique, génération annonce IA streaming, géocodage Nominatim, validation SIRET

CP4 | Contribuer à la gestion d'un projet informatique

Itérations V1/V2/V3 documentées, Git (branches par version), GitHub Actions CI/CD, planning Gantt

Bloc 2 — Concevoir et développer une application sécurisée en couches

CP5 | Analyser les besoins et maquetter une application

Analyse terrain (agents immo indépendants, 30-45 min/annonce), diagramme use case (15 UC, 3 acteurs), maquettes Figma

CP6 | Définir l'architecture logicielle d'une application

Architecture 3 couches (UI / API Routes / Données), isolation multi-tenant par ligne (agentId + agencyId), diagramme de séquence

CP7 | Concevoir et mettre en place une base de données relationnelle

Modélisation Merise MCD/MLD, 6 entités (Agency, Agent, Client, Property, VerificationToken, PendingRegistration), migrations Prisma

CP8 | Développer des composants d'accès aux données SQL et NoSQL

Prisma v7 : requêtes filtrées par agentId (isolation), nested write atomique (Agency + Agent), DELETE RETURNING (token)

Bloc 3 — Préparer le déploiement d'une application sécurisée

CP9 | Préparer et exécuter les plans de tests d'une application

29 tests : 7 unitaires Vitest (buildListingPrompt), 18 intégration (API routes + BDD réelle via tunnel SSH), 4 E2E Playwright (parcours complets avec agent isolé)

CP10 | Préparer et documenter le déploiement d'une application

Documentation VPS : PostgreSQL systemd, Node.js nvm, PM2, Nginx reverse proxy, gestion .env, checklist de mise en production

CP11 | Contribuer à la mise en production dans une démarche DevOps

GitHub Actions : tests → build → déploiement SSH conditionnel (if: success()), secrets GitHub Actions, pm2 restart automatique

1. Présentation du projet

1.1 Contexte et problématique

Promptimo est un projet personnel développé dans le cadre de mon apprentissage. L'idée naît d'un constat : les agents immobiliers indépendants passent en moyenne 30 à 45 minutes à rédiger chaque annonce manuellement, gèrent leurs contacts dans des outils non adaptés (Excel, carnet papier), et jonglent entre plusieurs applications sans vue d'ensemble sur leur portefeuille.

La problématique centrale est la suivante : comment permettre à un agent immobilier indépendant de centraliser la gestion de son activité (clients, biens, annonces) et de générer des annonces professionnelles en quelques secondes grâce à l'IA ?

1.2 Objectifs du projet

- Gestion du portefeuille clients et biens : CRUD complet avec association client ↔ bien
- Génération automatique d'annonces : prompt structuré envoyé au modèle Anthropic, réponse en streaming temps réel, sauvegarde en base
- Visualisation cartographique : carte Leaflet, géocodage automatique, repositionnement des pins par drag-and-drop
- Inscription avec paiement : formulaire en ligne, Stripe Checkout, activation par magic link

1.3 Périmètre fonctionnel

- Module authentification : magic link sans mot de passe (NextAuth v5), inscription Stripe
- Module clients : CRUD, filtres par type (vendeur, acheteur, bailleur, locataire)
- Module biens : CRUD avec géocodage automatique Nominatim, équipements, notes
- Module annonces IA : génération SSE streaming, régénération à la demande
- Module carte : marqueurs personnalisés par type, popups, déplacement de pins
- Hors périmètre V1/V2 : messagerie, agenda, rapports statistiques, multi-agents par agence

1.4 Contraintes

- Développement en solo, en parallèle de l'alternance
 - Budget infra limité : VPS Ionos S+ (1 vCPU, 2 Go RAM)
 - Déploiement sans conteneurisation (Docker) pour garder le contrôle total sur le serveur
 - Date limite : juillet 2026 (examen CDA RNCP 37873)
-

2. Gestion du projet

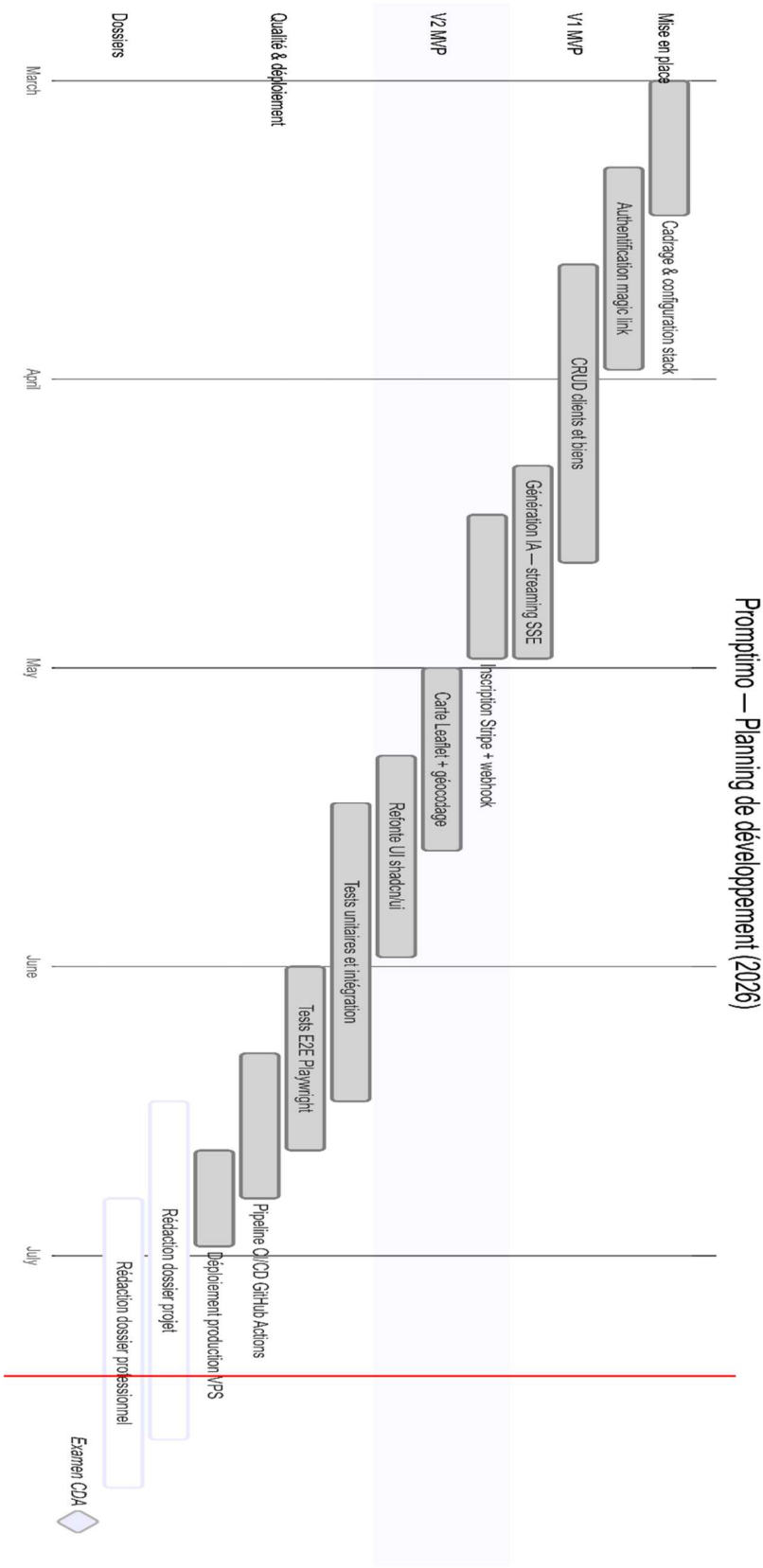
2.1 Méthode de développement

Le projet a été développé en itérations fonctionnelles successives :

- V1 MVP : authentification, gestion clients/biens, génération IA de base
- V2 MVP : carte interactive, webhooks Stripe, refonte UI, géocodage, tests automatisés, CI/CD
- V3 (prévue) : plans groupe, multi-agents, tableau de bord analytique

La gestion du code source repose sur Git (GitHub). Chaque version est développée sur une branche dédiée (ex. : V2_MVP) puis intégrée sur main. À partir de la V2, tout développement se fait directement sur main avec déploiement continu via CI/CD.

2.2 Planning Mermdaid



En cas de difficulté de lecture le SVG est téléchargeable : <https://www.transfERNOW.net/dl/PromptinoDevelopmentSVG>

3. Spécifications fonctionnelles

3.1 Acteurs

- Agent immobilier (utilisateur principal) : accède à l'application après authentification, gère son portefeuille
- Futur agent (visiteur) : soumet le formulaire d'inscription, effectue le paiement Stripe, reçoit le magic link d'activation
- Stripe (acteur système) : notifie l'application via webhook lors du paiement confirmé

3.2 Fonctionnalités principales

Authentification et inscription

- Formulaire d'inscription avec validation : SIRET (algorithme de Luhn + API INSEE), email, téléphone
- Paiement via Stripe Checkout (49 €/mois plan solo, 149 €/mois plan agence)
- Activation par magic link envoyé par email (Resend), token validé côté serveur
- Connexion sans mot de passe, session JWT chiffrée (JWE)

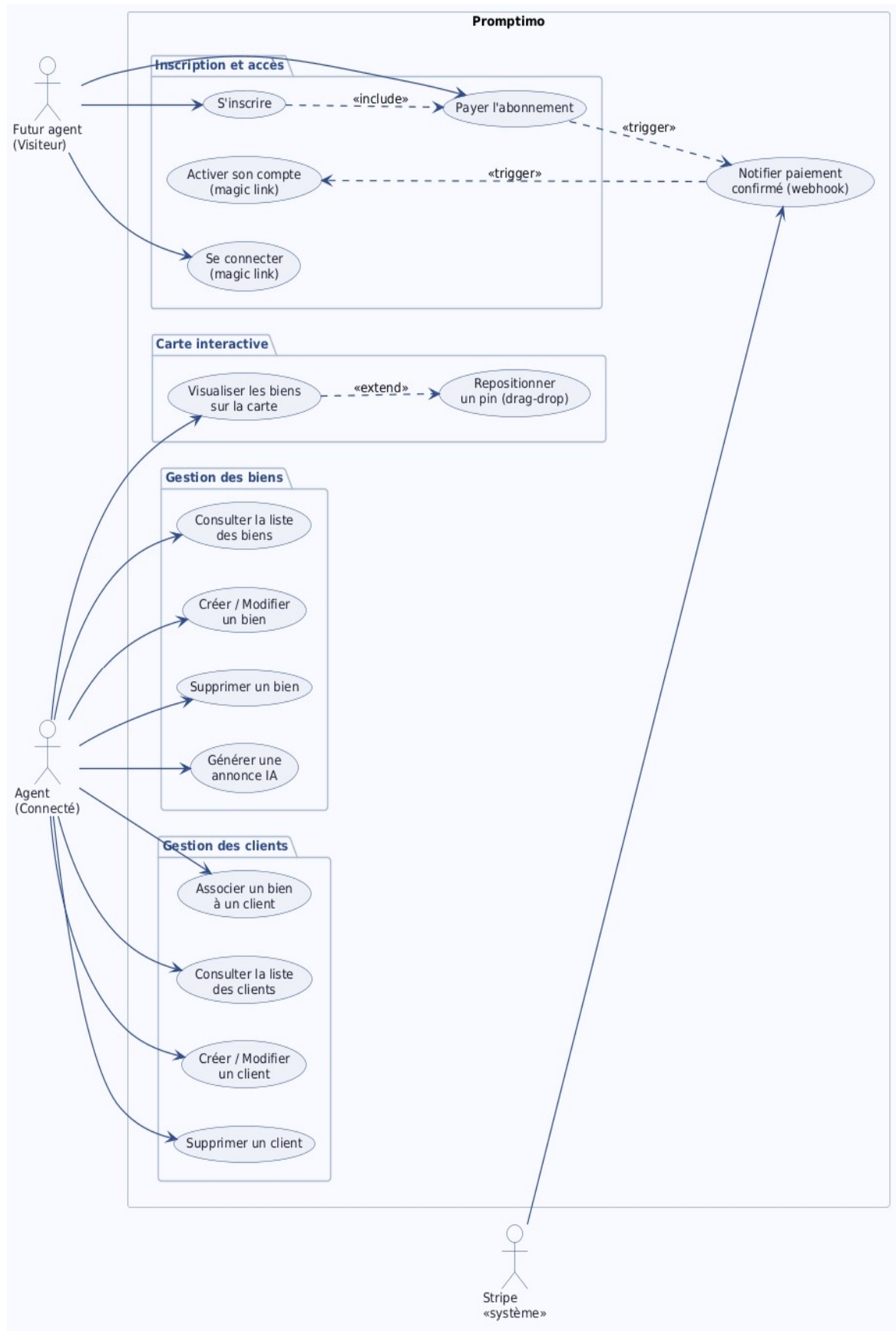
Gestion clients et biens

- CRUD clients avec filtres par type de client
- CRUD biens avec géocodage automatique et liste d'équipements
- Association client ↔ bien (un vendeur/bailleur peut être lié à un ou plusieurs biens)

Annonces IA et carte

- Génération d'annonce en un clic : prompt structuré, streaming SSE, sauvegarde et régénération
- Carte Leaflet avec marqueurs colorés par type, popups, repositionnement drag-and-drop

3.3 Diagramme de cas d'utilisation



4. Conception technique

4.1 Stack technique

- Framework : Next.js 16 (App Router) — rendu hybride RSC + Client Components
- Langage : TypeScript (strict mode)
- UI : React 19, Tailwind CSS v4, shadcn/ui (composants base-ui accessibles)
- ORM : Prisma v7 avec adaptateur pg (PostgreSQL natif)
- Base de données : PostgreSQL 18
- Authentification : NextAuth v5 (beta), stratégie JWT/JWE, provider Resend (magic link)
- Paiements : Stripe SDK v22 — Checkout Sessions + Webhooks HMAC-SHA256
- IA : Anthropic SDK — claude-sonnet-4-20250514, streaming SSE (ReadableStream)
- Carte : Leaflet 1.9 + react-leaflet, tuiles OpenStreetMap, géocodage Nominatim
- Tests : Vitest (unitaires + intégration) + Playwright (E2E, chromium)
- CI/CD : GitHub Actions, déploiement SSH sur VPS
- Infra : VPS Ionos S+ Ubuntu 26.04, PM2 (process manager), Nginx (reverse proxy)

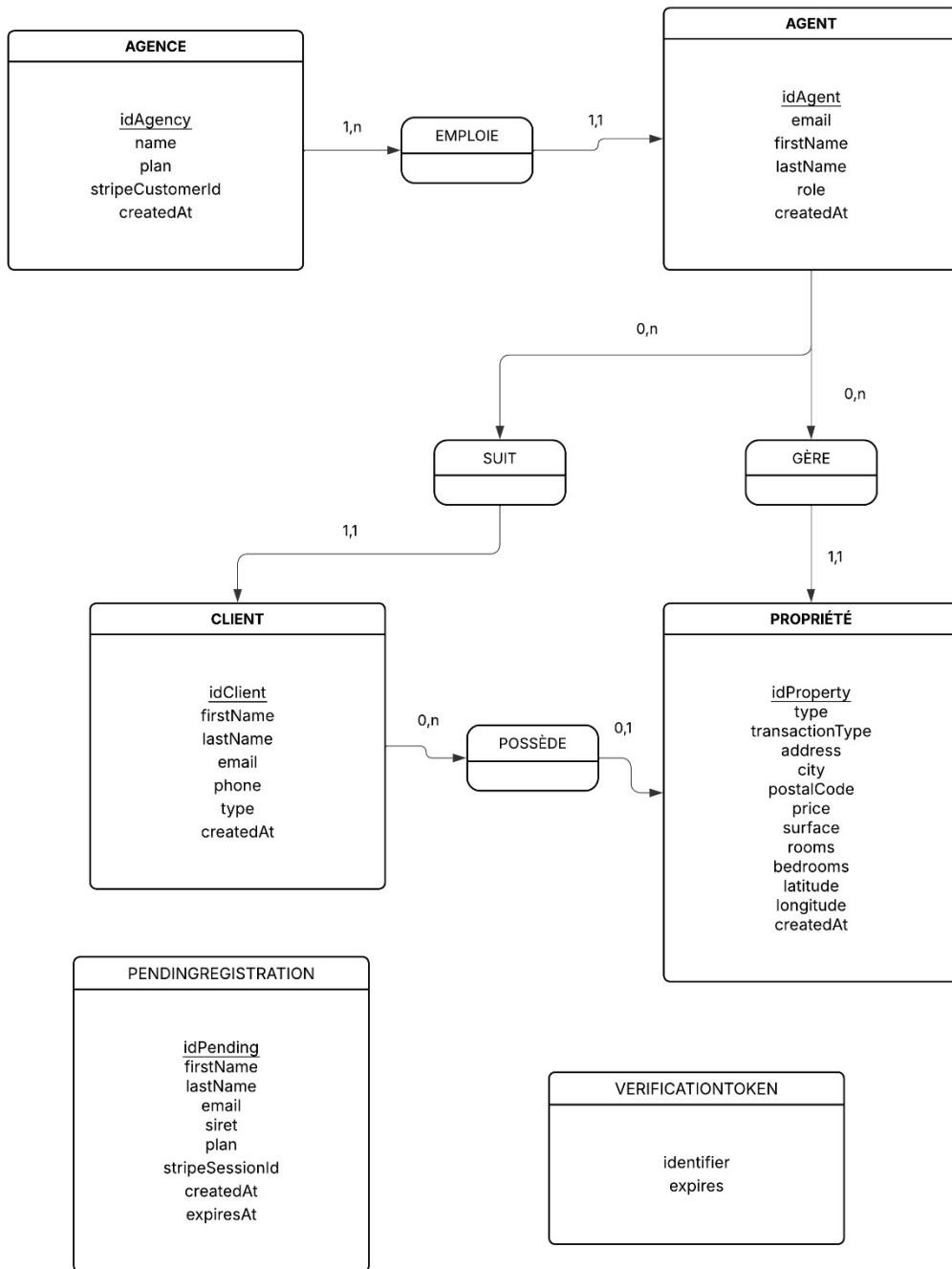
4.2 Architecture en couches

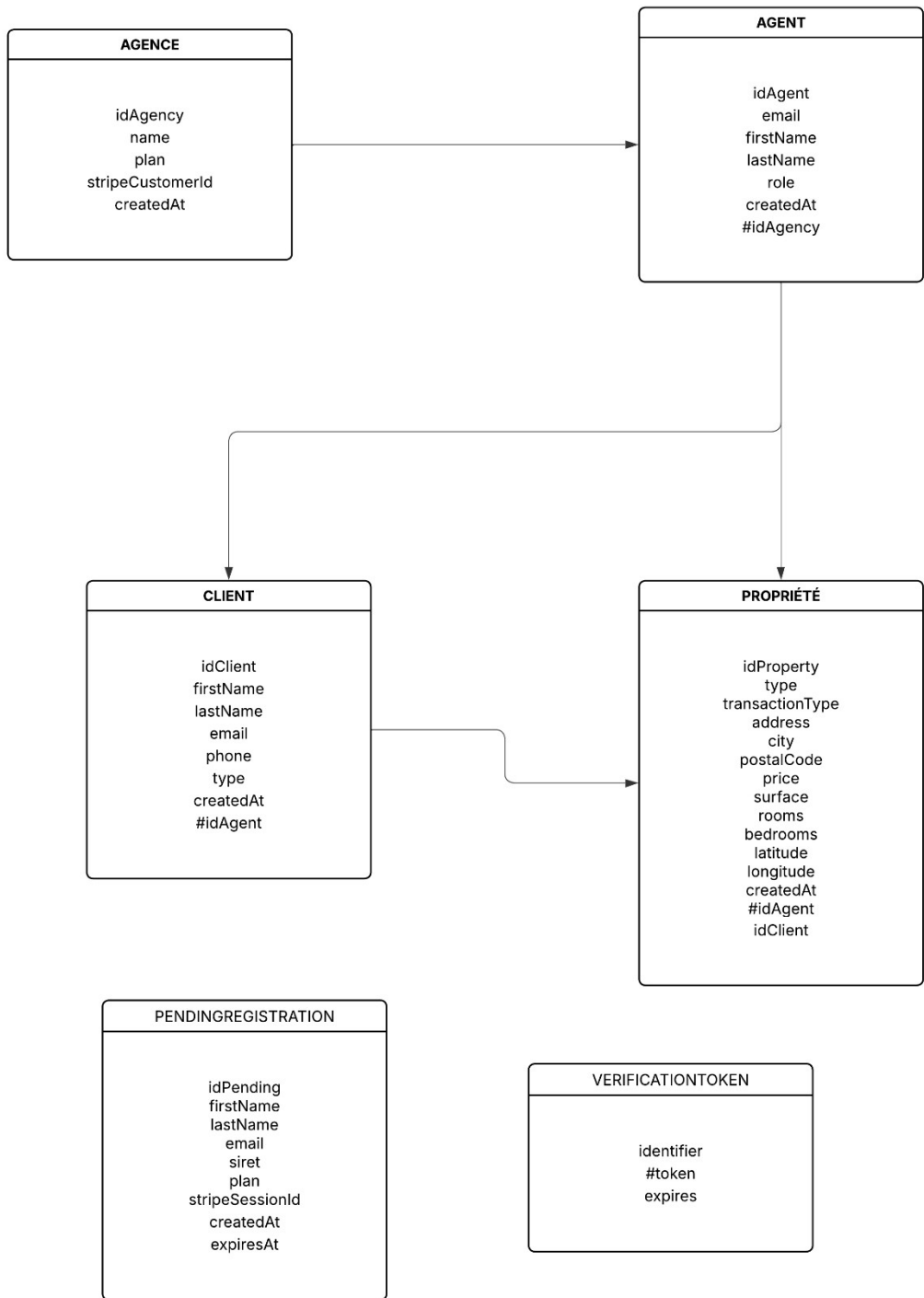
- Couche présentation (UI) : React Server Components pour les lectures directes Prisma, Client Components pour les interactions. Organisation par route sous `src/app/(protected)/`.
- Couche API (API Routes) : routes Next.js sous `src/app/api/`. Mutations, validation Zod, vérification session JWT, appels services tiers (Stripe, Anthropic, Resend).
- Couche données : singleton Prisma (`src/lib/prisma.ts`). Toutes les requêtes BDD passent par Prisma ORM. Migrations versionnées via `prisma migrate`.

4.3 Modèle de données

Le schéma relationnel comprend 6 entités : Agence (tenant principal), Agent (utilisateur, relié à une agence), Client (contact, relié à un agent), Propriété (bien immobilier, relié à un agent et optionnellement à un client), VerificationToken (magic link), PendingRegistration (inscription en attente de paiement).

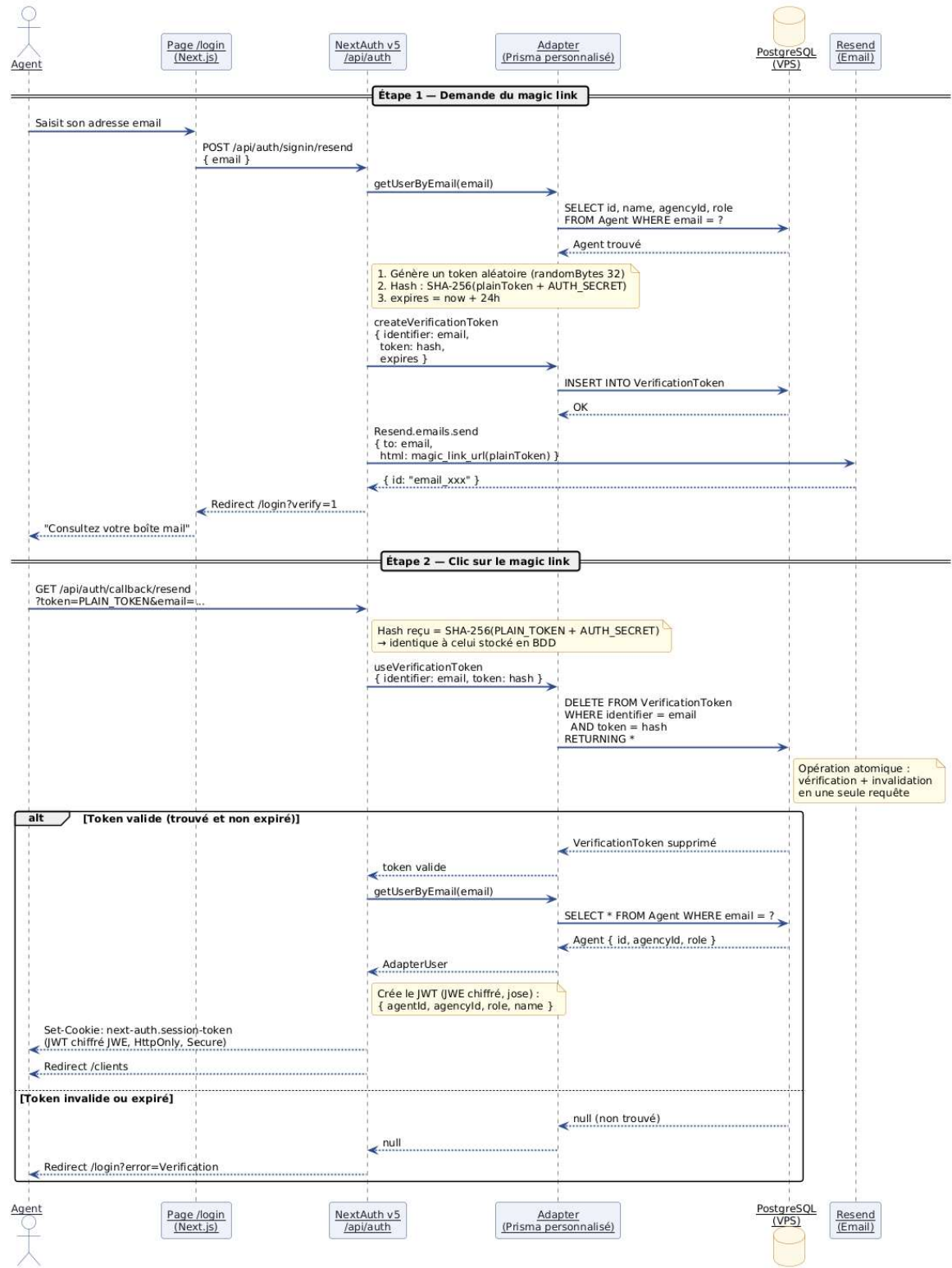
L'isolation multi-tenant est assurée par ligne : chaque Property et Client porte un `agentId`, lui-même lié à un `agencyId`. Aucune donnée d'une agence n'est accessible depuis une autre.





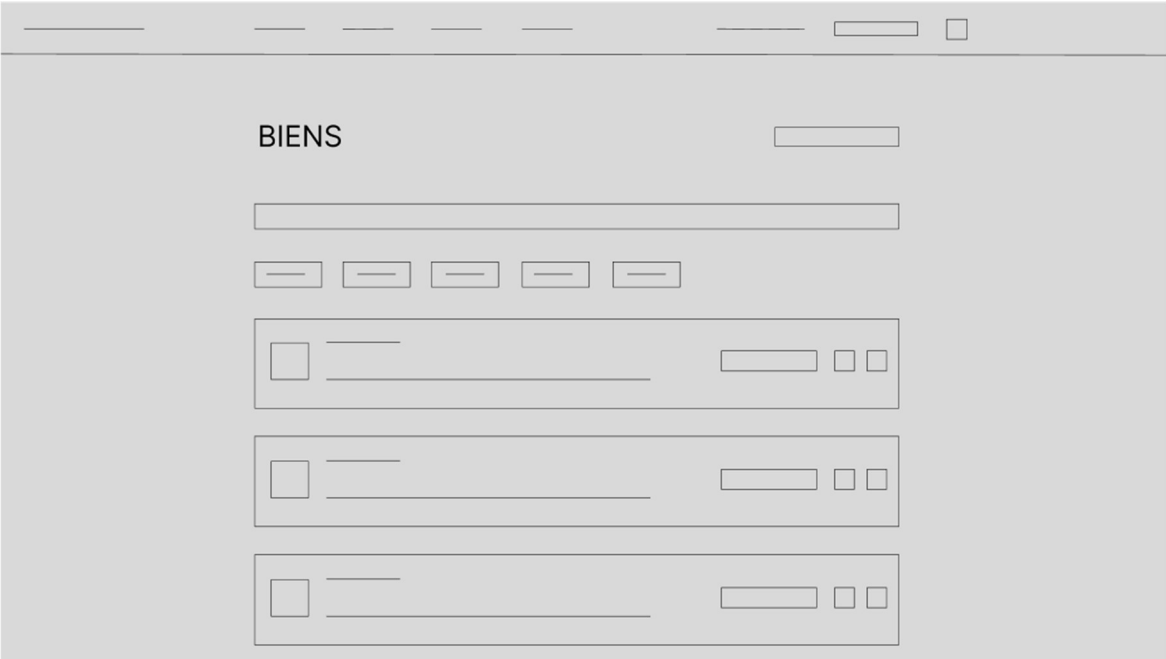
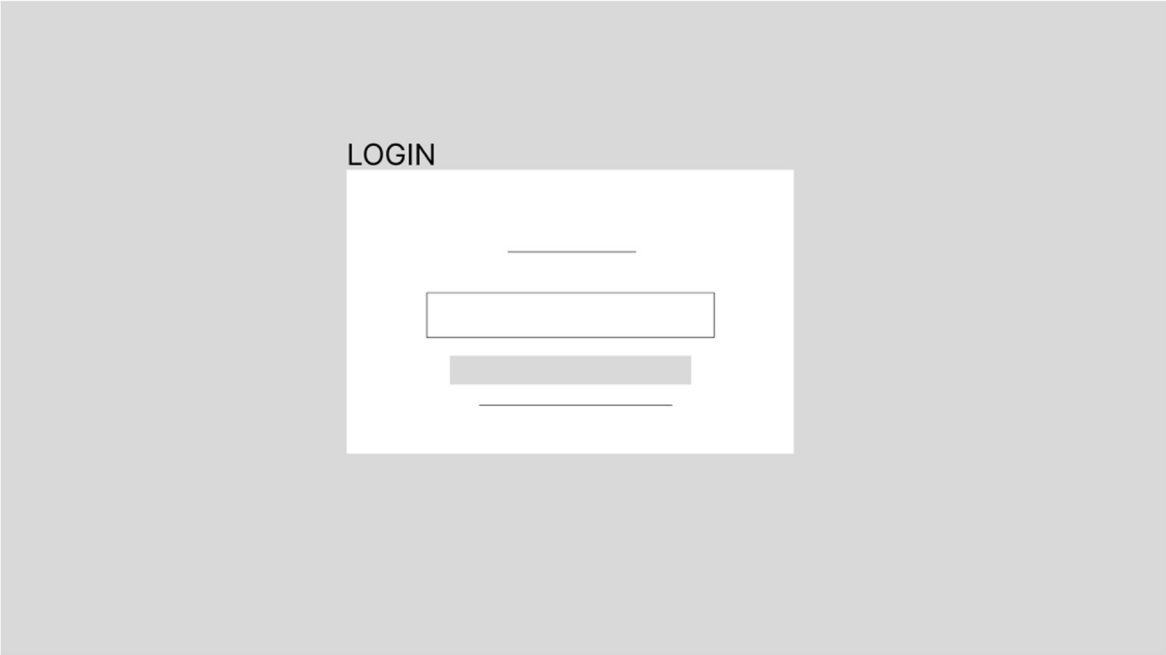
4.4 Diagrammes de séquence

Flux d'authentification — Magic Link (NextAuth v5)



5. Maquettes et interfaces

Les maquettes ont été réalisées sous Figma et couvrent les écrans principaux : page d'accueil, formulaire d'inscription, espace connecté (clients, biens, carte, annonce IA), navigation responsive (desktop et mobile).



[illegible]



Biens

30 biens

+ Nouveau bien

🔍 Rechercher par adresse, ville ou propriétaire...

Tous

Appartement

Maison

Studio

Villa

Loft

AP

Appartement 460 900 € FAI

📍 12 rue de la République, 13001 Marseille 🏠 116 m² 3 pièces Julie ROBERT

🏠 Annonce



LO

Loft 392 900 € FAI

📍 5 cours Belairce, 13001 Marseille 🏠 119 m² 4 pièces Baptiste MOREL

🏠 Annonce



ST

Studio 97 500 € FAI

📍 34 boulevard Garibaldi, 13001 Marseille 🏠 30 m² 1 pièce Sarah ROUX

🏠 Annonce



VI

Villa 2 310 € /mois CC

📍 10 rue de la République, 13001 Marseille 🏠 150 m² 5 pièces

🏠 Annonce



Type de bien

Type

Appartement

Maison

Studio

Villa

Loft

Local commercial

Terrain

Localisation & prix

Transaction

À vendre

À louer

Adresse

12 rue des Fleurs

Code postal

13001

Ville

Marseille

Prix — frais d'agence inclus (€)

0

Clients

30 clients

+ Nouveau client

Rechercher par nom ou email...

Tous

Vendeurs

Acheteurs

Bailleurs

Locataires

MM

Marie MARTIN

Bailleur

07 85 71 95 34

marie4@gmail.com

Email



TB

Thomas BERNARD

Acheteur

07 19 04 17 62

thomas.bernard@gmail.com

Email



ST

Sophie THOMAS

Bailleur

06 38 08 00 22

sophie.thomas@gmail.com

Email



NP

Nicolas PETIT

Vendeur

Email



Nouveau client

X

Prénom

Marie

Nom

DUPONT

Email (optionnel)

marie@exemple.fr

Téléphone (optionnel)

06 12 34 56 78

Type

Vendeur

Acheteur

Bailleur

Locataire

Notes (optionnel)

Budget, critères, situation,...

Enregistrer

Carte des biens

30 biens géolocalisés

Appartement

23 avenue de la Rose, Marseille

141400 €

Jonathan LE PLAT

Vente

42 m²

Villa

15 boulevard Barry, Marseille

1870 €

Jonathan LE PLAT

Location

154 m²

Appartement

36 avenue de Saint-Just, Marseille

910 €

Jonathan LE PLAT

Location

80 m²

Appartement

8 rue de la Joliette, Marseille

990 €

Jonathan LE PLAT

Location

90 m²

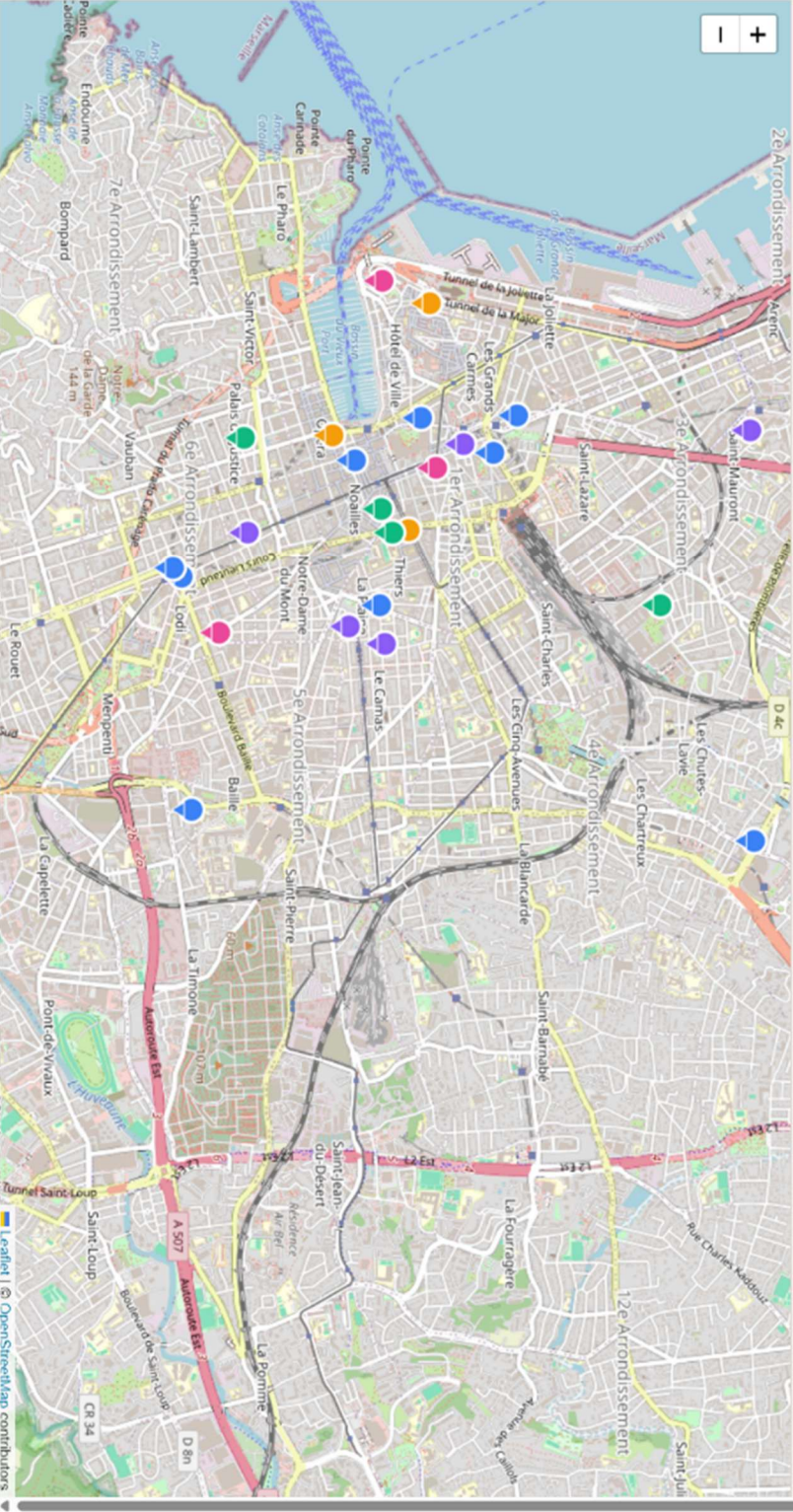
Villa

27 boulevard Chave, Marseille

593200 €

Vente

173 m²



✉️ **Generer un email**

Email professionnel prêt à envoyer en quelques secondes

Paramètres

Type d'email

Suivi de visite

Relance prospect

Nouveau bien

Offre reçue

Client

🔍 Rechercher par nom ou email...

Bien associé (optionnel)

Aucun bien sélectionné

Contexte supplémentaire (optionnel)

Précisions sur la situation, l'offre reçue, la dernière visite...

Le client a visité le bien jeudi dernier et semblait très intéressé...

📧 Générer l'email



Clients

30 clients

+ Nouveau client

Rechercher par nom ou email...

Tous

Vendeurs

Acheteurs

Bailleurs

Locataires

Marie MARTIN

Locataire

MM

07 69 18 71 56

Email



marie42@gmail.com

1 bien

Thomas
BERNARD

TB

Bailleur

06 86 80 56 33

Email



thomas.bernard@gmail.com

Sophie THOMAS

Vendeur

ST

06 68 84 20 87

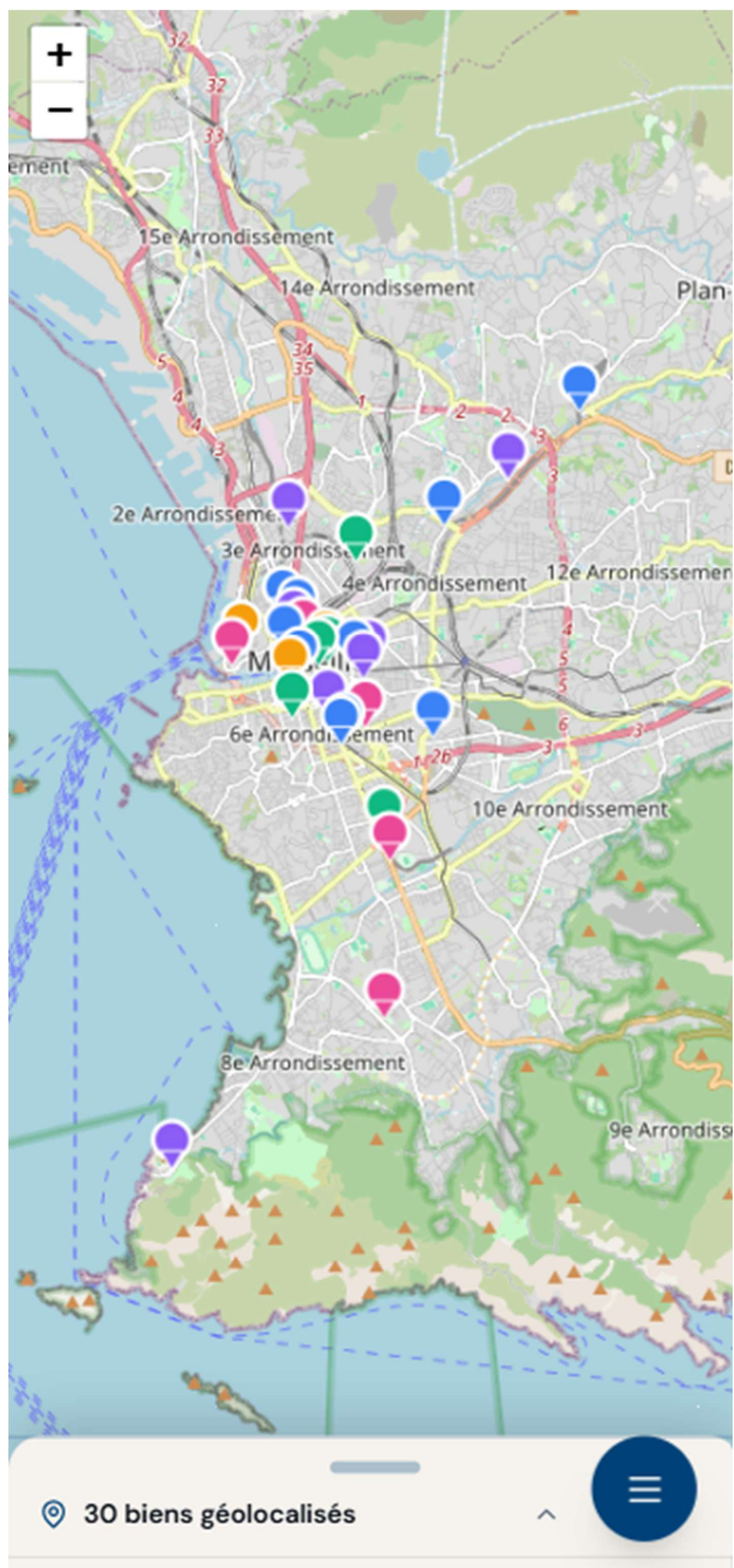
Email



thomas.s@gmail.com

1 bien





 Clients

 Biens

 Carte

 Nouvel email

 Collaborateurs

Thème



 Se déconnecter



6. Réalisations

6.1 Authentification (magic link + adaptateur personnalisé)

Promptimo implémente une authentification sans mot de passe via NextAuth v5. Le flux repose sur un lien magique envoyé par email (Resend) : l'utilisateur clique le lien, le token (stocké haché SHA-256 en BDD) est vérifié puis détruit, une session JWT est créée. La session est chiffrée en JWE et embarque les champs métier agentId, agencyId et role.

Un adaptateur Prisma personnalisé mappe le modèle Agent sur l'interface NextAuth, sans dupliquer les données dans une table User. La méthode createUser lève volontairement une erreur pour bloquer toute auto-inscription.

6.2 Inscription et paiement Stripe

Le flux d'inscription se déroule en trois étapes : soumission du formulaire (validation SIRET par Luhn + requête INSEE, unicité email), redirection vers Stripe Checkout, traitement du webhook checkout.session.completed.

À réception du webhook, l'application vérifie la signature HMAC-SHA256, crée atomiquement Agency + Agent via une nested write Prisma, génère un magic link signé et l'envoie par email. La table PendingRegistration est ensuite nettoyée.

6.3 Gestion clients et biens

Les pages sont rendues côté serveur (React Server Components) avec lectures directes Prisma. Les mutations (création, modification, suppression) passent par des API routes sécurisées par session JWT. L'association client ↔ bien est gérée via un PATCH sur la relation clientId de Property.

6.4 Génération d'annonces IA (streaming SSE)

La route /api/listings/stream construit un prompt structuré à partir des caractéristiques du bien, appelle l'API Anthropic en mode streaming et retransmet chaque fragment via ReadableStream. Le composant React lit ce flux avec getReader() et affiche le texte en temps réel avec un curseur clignotant. L'annonce complète est sauvegardée en base à la fin du stream.

6.5 Carte interactive (Leaflet + géocodage)

La vue carte utilise Leaflet avec des tuiles OpenStreetMap. L'adresse saisie est géocodée automatiquement via l'API Nominatim. Les marqueurs sont personnalisés par type de bien (couleur distincte). Une fonctionnalité de repositionnement par glisser-déposer permet d'ajuster la position d'un pin et sauvegarde les nouvelles coordonnées via un PATCH sur /api/map/properties/[id].

7. Sécurité

Authentification et sessions

- Magic link sans mot de passe : suppression totale de la surface d'attaque bruteforce
- Token de vérification haché SHA-256 avant stockage : compromis BDD ne suffit pas à régénérer le lien
- Session JWT chiffrée JWE (jose) : contenu illisible sans AUTH_SECRET
- Middleware Next.js : toutes les routes /protected/* vérifient la validité du JWT

Autorisation et isolation des données

- Chaque requête API vérifie session.user.id (agentId) : un agent ne peut accéder qu'à ses propres données
- Requêtes Prisma avec WHERE { id, agentId } : impossible d'accéder au bien d'un autre agent
- Isolation multi-tenant par ligne via agencyId sur toutes les entités

Validation et intégrité

- Validation Zod systématique à chaque entrée API : types, formats, valeurs autorisées
- Validation SIRET : algorithme de Luhn + appel API INSEE (sirene.entreprises.api.gouv.fr)
- Webhook Stripe : vérification HMAC-SHA256 via stripe.webhooks.constructEvent, rejet HTTP 400 si échec

Infrastructure et secrets

- Fichiers .env exclus du dépôt Git (.gitignore) dès le premier commit
 - Secrets CI/CD stockés dans GitHub Actions Secrets (VPS_SSH_KEY, DATABASE_URL_TEST, AUTH_SECRET)
 - .env de production créé directement sur le VPS via SSH, jamais versionné
-

8. Tests

8.1 Stratégie de tests

La stratégie couvre trois niveaux :

- Tests unitaires (Vitest) : validation de la fonction buildListingPrompt (construction du prompt IA) avec différents jeux de données
- Tests d'intégration (Vitest) : appels HTTP réels aux API routes Next.js avec vraie BDD PostgreSQL (VPS via tunnel SSH) et services externes mockés via vi.mock() (Stripe, Resend)
- Tests End-to-End (Playwright, chromium) : parcours utilisateur complets — session JWT pré-construite, agent de test isolé créé et supprimé par le global setup/teardown

Les 29 tests s'exécutent automatiquement via le pipeline GitHub Actions à chaque push sur main. Le déploiement est conditionné à leur réussite complète.

8.2 Table des tests

Résultats des 29 tests automatisés (7 unitaires, 18 d'intégration, 4 E2E) :

ID	Scénario de test	Type	Jeu de données / Préconditions	Résultat attendu	Statut
T-U-01	buildListingPrompt — champs obligatoires	Unitaire	type, address, city, price: 250000, surface: 65	Prompt contient type, adresse, ville, "65 m²"	✓ Passé
T-U-02	buildListingPrompt — formatage prix FR	Unitaire	price: 250000	"250 000" avec séparateur de milliers français	✓ Passé
T-U-03	buildListingPrompt — avec points forts	Unitaire	features: ["Terrasse", "Parking"]	Prompt contient "Terrasse" et "Parking"	✓ Passé
T-U-04	buildListingPrompt — sans points forts	Unitaire	features: []	Prompt ne contient pas "Points forts"	✓ Passé
T-U-05	buildListingPrompt — pièces et chambres	Unitaire	rooms: 3, bedrooms: 2	"Pièces : 3" et "Chambres : 2" dans le prompt	✓ Passé
T-U-06	buildListingPrompt — sans pièces	Unitaire	Base sans rooms ni bedrooms	Prompt ne contient pas "Pièces :"	✓ Passé
T-U-07	buildListingPrompt — instructions rédaction	Unitaire	Objet de base standard	Contient "SeLoger" et "150-200 mots"	✓ Passé
T-I-01	POST /api/register — champs manquants	Intégration	{ firstName: "" } — body incomplet	HTTP 400	✓ Passé
T-I-02	POST /api/register — emails différents	Intégration	email ≠ emailConfirm	HTTP 400	✓ Passé
T-I-03	POST /api/register — SIRET invalide	Intégration	siret: "1234" (< 14 chiffres)	HTTP 400	✓ Passé
T-I-04	POST /api/register — plan invalide	Intégration	plan: "premium"	HTTP 400	✓ Passé
T-I-05	POST /api/register — email déjà Agent	Intégration	Agent existant en BDD avec le même email	HTTP 409, body.error contient "déjà utilisée"	✓ Passé
T-I-06	POST /api/register — PendingRegistration actif	Intégration	PendingRegistration non expiré en BDD	HTTP 409	✓ Passé
T-I-07	POST /api/register — PendingRegistration expiré	Intégration	PendingRegistration expiré en BDD	HTTP 201 (email réutilisable)	✓ Passé
T-I-08	POST /api/register — création Pending	Intégration	Payload valide (plan solo, email unique)	PendingRegistration créé, stripeSessionId présent	✓ Passé
T-I-09	POST /api/register — retourne checkoutUrl	Intégration	Payload valide	HTTP 201, json.checkoutUrl = URL Stripe Checkout	✓ Passé
T-I-10	POST /api/register — montant plan solo	Intégration	plan: "solo"	Stripe : unit_amount = 4900, currency = "eur"	✓ Passé

ID	Scénario de test	Type	Jeu de données / Préconditions	Résultat attendu	Statut
T-I-11	POST /api/register — montant plan agence	Intégration	plan: "agence"	Stripe : unit_amount = 14900	✓ Passé
T-I-12	POST /api/webhooks/stripe — signature invalide	Intégration	Signature HMAC-SHA256 incorrecte	HTTP 400	✓ Passé
T-I-13	POST /api/webhooks/stripe — event inconnu	Intégration	type: "payment_intent.created"	HTTP 200, aucune agence créée	✓ Passé
T-I-14	POST /api/webhooks/stripe — ID absent	Intégration	metadata.pendingRegistrationId manquant	HTTP 200, aucune agence créée	✓ Passé
T-I-15	POST /api/webhooks/stripe — création Agency + Agent	Intégration	checkout.session.completed + PendingRegistration valide	Agency et Agent créés, Agent.role = "agent"	✓ Passé
T-I-16	POST /api/webhooks/stripe — VerificationToken	Intégration	checkout.session.completed + PendingRegistration valide	Token créé en BDD, expiration dans 24h	✓ Passé
T-I-17	POST /api/webhooks/stripe — email activation	Intégration	checkout.session.completed + PendingRegistration valide	Resend.send() appelé, destinataire correct, objet contient "Activez"	✓ Passé
T-I-18	POST /api/webhooks/stripe — nettoyage	Intégration	checkout.session.completed + PendingRegistration valide	PendingRegistration supprimé de la BDD	✓ Passé
T-E-01	Page /clients — affichage du titre	E2E	Agent authentifié (JWT), navigation GET /clients	Heading "Clients" visible	✓ Passé
T-E-02	Création client via dialog	E2E	Clic "Nouveau client", form: Marie DUPONT, type Vendeur, soumission	Toast "Client ajouté !" + client visible dans la liste	✓ Passé
T-E-03	Création d'un bien immobilier	E2E	Form: Appartement, À vendre, 12 rue des Fleurs, 13001, 250000 €, 75 m²	Redirection /properties/[id]/listing, h1 contient "Appartement" + "Marseille"	✓ Passé
T-E-04	Génération annonce IA (SSE mocké)	E2E	Route /api/listings/stream interceptée, clic "Générer"	Texte du mock affiché dans la page	✓ Passé

9. Déploiement

9.1 Infrastructure

- VPS Ionos S+ (Ubuntu 26.04) : PostgreSQL 18 (systemd), Node.js 24 (nvm)
- PM2 : process manager avec redémarrage automatique au crash ou reboot
- Nginx : reverse proxy port 80 → port 3000 de l'application Next.js

L'application est accessible à l'adresse <http://82.165.114.226>. Mise en place d'un nom de domaine et certificat SSL/TLS (Let's Encrypt / Certbot) prévue.

9.2 Pipeline CI/CD (GitHub Actions)

Le workflow `.github/workflows/ci-cd.yml` se déclenche à chaque push sur main :

- Phase 1 — Tests : installation dépendances, génération client Prisma, configuration SSH, tunnel BDD, reconstruction `.env.test`, installation Chromium, exécution des 29 tests
- Phase 2 — Déploiement (conditionnel) : SSH sur le VPS, git pull, pnpm build, pm2 restart promptimo
- En cas d'échec d'un test : déploiement annulé (if: success()) — la production est protégée

9.3 Environnements

- Développement : `.env.local`, BDD via tunnel SSH, clés Stripe en mode test
- Test CI/CD : `.env.test` reconstruit depuis GitHub Secrets, même tunnel SSH, services tiers mockés
- Production : `.env` stocké sur le VPS (jamais versionné), clés Stripe en mode live

10. Veille technologique

Sujet : L'authentification sans mot de passe (passwordless) — sécurité et adoption (*directement lié à l'implémentation du magic link dans Promptimo*)

Introduction

La gestion des mots de passe représente l'une des principales causes de compromission de comptes : réutilisation, phishing, fuites de bases de données. Face à ces risques, une tendance de fond émerge depuis 2019 : **l'authentification sans mot de passe (passwordless)**. Elle couvre un ensemble de mécanismes permettant d'identifier un utilisateur sans lui demander de mémoriser un secret statique.

1. Les mécanismes passwordless existants

1.1 Magic link (lien magique par email) Un token à usage unique est envoyé par email. L'utilisateur clique le lien, le token est vérifié et détruit, une session est ouverte. Simple à implémenter, compatible avec tous les appareils, mais dépend de la sécurité de la boîte email.

1.2 Passkeys (FIDO2/WebAuthn) Standard W3C publié en 2019, largement adopté depuis 2022. L'authentification s'appuie sur une paire de clés cryptographiques (clé privée sur le device, clé publique sur le serveur) et une preuve biométrique locale (Face ID, empreinte, PIN). Aucun secret ne transite sur le réseau. Apple, Google et Microsoft ont intégré les passkeys nativement en 2023. GitHub les propose depuis mai 2023.

1.3 OTP (One-Time Password) par SMS ou TOTP Code à 6 chiffres valable 30 secondes (TOTP, RFC 6238). Utilisé comme second facteur (2FA) plutôt que facteur unique. Le SMS OTP est vulnérable au SIM swapping.

2. Comparaison sécurité

Mécanisme	Résistance phishing	Résistance brute force	Dépendance externe
Mot de passe seul	Faible	Dépend de la politique	Non
Magic link email	Moyenne (dépend email)	Forte (usage unique)	Fournisseur email
Passkey (FIDO2)	Très forte	Très forte	Non (device)
OTP TOTP	Forte	Forte	Non
OTP SMS	Faible (SIM swap)	Forte	Opérateur télécom

3. Application dans Promptimo

Promptimo implémente le magic link via NextAuth v5 et Resend. Le token généré (crypto.randomBytes(32)) est haché en SHA-256 avant stockage en base de données — ainsi, une compromission de la base ne permet pas de rejouer le lien. La durée de validité est fixée à 24 heures. Le token est détruit immédiatement après usage (opération atomique DELETE ... RETURNING). Pour une version future, la migration vers les passkeys serait pertinente : NextAuth v5 intègre un provider WebAuthn expérimental depuis la beta 5.0. Cela éliminerait la dépendance au fournisseur email (Resend) et renforcerait la résistance au phishing.

4. Sources et veille

- FIDO Alliance — <https://fidoalliance.org/passkeys/> — état de l'adoption mondiale
- W3C WebAuthn Level 3 — <https://www.w3.org/TR/webauthn-3/> — spécification technique
- ANSSI — Recommandations relatives à l'authentification (2021, mise à jour 2023)
- NextAuth.js changelog — intégration WebAuthn v5 (beta)
- GitHub Security Blog — déploiement des passkeys (mai 2023)

11. Bilan et perspectives

Promptimo est une application full-stack complète, déployée et en production, qui couvre l'ensemble du cycle de vie d'un projet logiciel : de la conception à la mise en production, en passant par les tests automatisés et le déploiement continu.

Compétences développées

- Maîtrise de Next.js 16 App Router et de l'architecture hybride RSC / Client Components
- Intégration de services tiers réels (Stripe, Anthropic, Resend) avec tests et sécurisation
- Mise en place d'un pipeline CI/CD opérationnel avec tests automatisés sur BDD réelle
- Gestion complète d'un VPS : installation, sécurisation, déploiement, monitoring PM2
- Architecture en couches, isolation multi-tenant, sécurité JWT/JWE

Perspectives V3

- Plans tarifaires groupe : plusieurs agents par agence, rôle admin avec gestion des membres
- Tableau de bord analytique : statistiques de portefeuille, taux de conversion
- Nom de domaine + HTTPS (Let's Encrypt / Certbot)